Task 1 — Load Data and Build the BST

1(a) — Load the airport file

```
FUNCTION loadAirports(filename)
    OPEN airports.dat
    A ← empty list
    skipped ← 0
    WHILE another line can be read do
        line ← next line from file
        fields ← split line at commas outside quotation marks
        code ← IATA field (field 5)
        IF code = "\N" or code does not have 3 letters then
            skipped ← skipped + 1
        ELSE
            airport ← record with code, name, city, country, ICAO, latitude, longitude
            append airport to A
        END IF
    END WHILE
    CLOSE file
    RETURN A, skipped
END FUNCTION
```

1(b) — Insert an airport (iterative)

```
FUNCTION insert(airport)
    newNode ← new node containing airport
    IF root = NULL then
        root ← newNode
        RETURN True
    END IF
    current ← root
    WHILE current ≠ NULL do
        IF airport.code = current.code then
            RETURN False
        ELSE IF airport.code < current.code then
            IF current.left = NULL then
                current.left ← newNode
                RETURN True
            END IF
            current ← current.left
        ELSE
            IF current.right = NULL then
                current.right ← newNode
                RETURN True
            END IF
            current ← current.right
        END IF
    END WHILE
END FUNCTION
```

1(b) — Build the file-order tree

```
FUNCTION buildTree(airports)
    root ← NULL
    duplicates ← 0
    n ← size(airports)
    FOR i ← 0 to n – 1 do
        added ← INSERT(airports[i])
        IF added = False then
            duplicates ← duplicates + 1
            PRINT "Duplicate code:", airports[i].code
        END IF
    END FOR
    RETURN root, duplicates
END FUNCTION
```

1(c) — Calculate height

```
FUNCTION calculateHeight(node)
    IF node = NULL then
        RETURN –1
    END IF
    leftHeight ← HEIGHT(node.left)
    rightHeight ← HEIGHT(node.right)
    RETURN 1 + max(leftHeight, rightHeight)
END FUNCTION
```

1(c) — Calculate average depth

```
FUNCTION averageDepth()
    IF root = NULL then
        RETURN 0
    END IF
    totalDepth ← 0
    count ← 0
    queue ← [(root, 0)]
    WHILE queue is not empty do
        (node, depth) ← remove front of queue
        totalDepth ← totalDepth + depth
        count ← count + 1
        IF node.left ≠ NULL then
            add (node.left, depth + 1) to queue
        END IF
        IF node.right ≠ NULL then
            add (node.right, depth + 1) to queue
        END IF
    END WHILE
    RETURN totalDepth / count
END FUNCTION
```

Task 2 — Search for Airport Codes

2(a) — Find an airport

```
FUNCTION find(code)
    current ← root
    comparisons ← 0
    WHILE current ≠ NULL do
        PRINT current.code as next code on the path
        comparisons ← comparisons + 1
        IF code = current.code then
            PRINT airport name, city, country, ICAO, latitude, longitude
            PRINT comparisons
            RETURN current.airport
        ELSE IF code < current.code then
            current ← current.left
        ELSE
            current ← current.right
        END IF
    END WHILE
    PRINT "Airport not found" and comparisons
    RETURN NULL
END FUNCTION
```

2(b) — Minimum and maximum codes

```
FUNCTION findMinMax()
    small ← root
    WHILE small.left ≠ NULL do
        small ← small.left
    END WHILE
    large ← root
    WHILE large.right ≠ NULL do
        large ← large.right
    END WHILE
    PRINT small.code and large.code
END FUNCTION
```

2(b) — Test the search cases

```
FUNCTION testSearchCases()
    find(root.code)
    find(code of a leaf node)
    find(code of a deep node)
    find("XYZ")
    (smallest, largest) ← MIN-MAX(root)
    find(code smaller than smallest)
    find(code larger than largest)
    PRINT smallest and largest
```

```
END FUNCTION
```

Task 3 — Remove Airports

3(a) — Delete an airport

```
FUNCTION remove(code)
    parent ← NULL
    current ← root
    WHILE current ≠ NULL and current.code ≠ code do
        parent ← current
        IF code < current.code then
            current ← current.left
        ELSE
            current ← current.right
        END IF
    END WHILE
    IF current = NULL then
        PRINT "Airport not found"
        RETURN False
    END IF
    IF current.left ≠ NULL and current.right ≠ NULL then
        successorParent ← current
        successor ← current.right
        WHILE successor.left ≠ NULL do
            successorParent ← successor
            successor ← successor.left
        END WHILE
        current.airport ← successor.airport
        parent ← successorParent
        current ← successor
    END IF
    IF current.left ≠ NULL then
        child ← current.left
    ELSE
        child ← current.right
    END IF
    IF parent = NULL then
        root ← child
    ELSE IF parent.left = current then
        parent.left ← child
    ELSE
        parent.right ← child
    END IF
    DELETE current
    PRINT "Airport deleted"
    RETURN True
END FUNCTION
```

3(b) — Test all deletion cases

```
FUNCTION testDeletionCases()
    FOR each case in [leaf, only left child, only right child, two children, root] do
        find a code matching this case in the current tree
        PRINT top three levels in level order
        REMOVE(code)
        PRINT top three levels in level order
        PRINT VALIDATE(root, no lower bound, no upper bound)
    END FOR
END FUNCTION
```

3(c) — Test a new airport

```
FUNCTION testNewAirport()
    airport ← new Airport with code "QQQ", name "Test International Airport", city "Boston"
    INSERT(airport)
    FIND("QQQ")
    REMOVE("QQQ")
    FIND("QQQ")
    PRINT VALIDATE(root, no lower bound, no upper bound)
END FUNCTION
```

Task 4 — Traversals, Save/Load and Validation

4(a) — List airports in alphabetical order (inorder)

```
FUNCTION inorder(node)
    IF node = NULL then
        RETURN
    END IF
    INORDER(node.left)
    PRINT node.airport
    INORDER(node.right)
END FUNCTION
```

4(a) — Check that the output is sorted

```
FUNCTION checkSorted(codes)
    n ← size(codes)
    FOR i ← 1 to n – 1 do
        IF codes[i – 1] >= codes[i] then
            RETURN False
        END IF
    END FOR
    RETURN True
END FUNCTION
```

4(b) — Save codes in preorder

```
FUNCTION savePreorder(node, file)
```

```
    IF node = NULL then
        RETURN
    END IF
    WRITE node.code to file
    PREORDER-SAVE(node.left, file)
    PREORDER-SAVE(node.right, file)
END FUNCTION
```

4(b) — Reload from saved preorder

```
FUNCTION loadPreorder(filename, airportRecords)
    newRoot ← NULL
    OPEN saved file
    WHILE another code can be read do
        code ← next code from file
        airport ← look up full Airport record FOR code
        IF airport exists then
            INSERT-INTO-NEW-TREE(airport)
        END IF
    END WHILE
    CLOSE file
    RETURN newRoot
END FUNCTION
```

4(b) — Print level order

```
FUNCTION levelOrder(root)
    IF root = NULL then
        RETURN empty list
    END IF
    queue ← [root]
    result ← empty list
    WHILE queue is not empty do
        node ← remove front of queue
        append node.code to result
        IF node.left ≠ NULL then
            add node.left to queue
        END IF
        IF node.right ≠ NULL then
            add node.right to queue
        END IF
    END WHILE
    RETURN result
END FUNCTION
```

4(b) — Check the reloaded shape

```
FUNCTION compareTreeShape(original, reloaded)
    oldHeight ← HEIGHT(original.root)
    newHeight ← HEIGHT(reloaded.root)
```

```
    oldLevels ← LEVEL-ORDER(original.root)
    newLevels ← LEVEL-ORDER(reloaded.root)
    IF oldHeight = newHeight and oldLevels = newLevels then
        RETURN True
    ELSE
        RETURN False
    END IF
END FUNCTION
```

4(c) — Validate the BST with bounds

```
FUNCTION validate(node, low, high)
    IF node = NULL then
        RETURN True
    END IF
    IF lo exists and node.code <= lo then
        RETURN False
    END IF
    IF hi exists and node.code >= hi then
        RETURN False
    END IF
    leftOK ← VALIDATE(node.left, lo, node.code)
    rightOK ← VALIDATE(node.right, node.code, hi)
    RETURN leftOK and rightOK
END FUNCTION
```

Task 5 — Range Queries

5 — Range search with subtree pruning

```
FUNCTION range(fromCode, toCode)
    visited ← 0
    matches ← 0
    RANGE-HELPER(root, FROM, TO)
    PRINT matches and visited

    FUNCTION RANGE-HELPER(node, FROM, TO)
        IF node = NULL then
            RETURN
        END IF
        visited ← visited + 1
        IF FROM < node.code then
            RANGE-HELPER(node.left, FROM, TO)
        END IF
        IF FROM <= node.code and node.code <= TO then
            PRINT node.airport
            matches ← matches + 1
        END IF
        IF node.code < TO then
            RANGE-HELPER(node.right, FROM, TO)
```

```
        END IF
    END FUNCTION
END FUNCTION
```

Task 6 — The Shape Experiment

6(a) — Build trees in different orders

```
FUNCTION buildDifferentTrees(airports)
    fileTree ← new empty BST
    FOR i ← 0 to size(A) – 1 do
        INSERT-INTO(fileTree, A[i])
    END FOR
    S ← copy A sorted by IATA code
    sortedTree ← new empty BST
    FOR i ← 0 to size(S) – 1 do
        INSERT-INTO(sortedTree, S[i])
    END FOR
    balancedTree.root ← BUILD-BALANCED(S, 0, size(S) – 1)
    R ← randomly shuffled copy of A
    randomTree ← new empty BST
    FOR i ← 0 to size(R) – 1 do
        INSERT-INTO(randomTree, R[i])
    END FOR
END FUNCTION
```

6(a) — Build a balanced BST using the middle element

```
FUNCTION buildBalanced(airports, low, high)
        IF low > high then
            RETURN NULL
        END IF
        mid ← floor((low + high) / 2)
        node ← new node(A[mid])
        node.left ← BUILD-BALANCED(A, low, mid – 1)
        node.right ← BUILD-BALANCED(A, mid + 1, high)
        RETURN node
END FUNCTION
```

6(b) — Collect tree statistics

```
FUNCTION treeStatistics(tree)
    h ← HEIGHT(root)
    avg ← AVERAGE-DEPTH(root)
    RETURN h, avg
END FUNCTION
```

6(c) — Time 10,000 searches

```
FUNCTION runExperiment(fileTree, sortedTree, balancedTree, randomTree, codes)
```

```
    queries ← 10000 randomly chosen codes from the valid code list
    FOR each tree in [fileTree, sortedTree, balancedTree, randomTree] do
        h ← HEIGHT(tree.root)
        avg ← AVERAGE-DEPTH(tree.root)
        start ← current timer value
        FOR i ← 0 to 9999 do
            SEARCH-WITHOUT-PRINTING(tree, queries[i])
        END FOR
        elapsed ← current timer value – start
        PRINT tree name, h, avg, elapsed
    END FOR
END FUNCTION
```